

Knowledge Representation and Reasoning

National University of Science and Technology
POLITEHNICA Bucharest

Department of Computer Science

Fall 2026

Adina Magda Florea

Lecture 3

Rule-based languages

SWRL

DROOLS

Rules and LLMs

1. Rule-based languages

- Knowledge is formulated in an “if-then” manner
- A rule specifies a condition and a consequence

if $cond_1 \wedge \ldots \wedge cond_n$
then conclusion(s) / perform action(s)
OR
when trigger
if $cond_1 \wedge \ldots \wedge cond_n$
then perform action(s)

LHS · left-hand side conditions

RHS · right-hand side conclusions / actions

- Such a statement is called production rule or simply rule
- Given by domain experts during knowledge acquisition

- Logical inference: Person(?x) ∧ teaches(?x, ?c) → TeachingStaff(?x)
- Operational rule: IF temperature exceeds a threshold THEN request cooling
- Not every if-then statement has the same semantics: deriving facts differs from executing actions

Four different roles of rules

1. INFERENCE

derive entailed facts from a knowledge base
(e.g., SWRL)

```
Person(?x) ∧ hasParent(?x, ?y) ∧
Woman(?y) -> hasMother(?x, ?y)
```

KB

```
:John rdf:type :Person .
:John :hasParent :Mary .
:Mary rdf:type :Woman .
```

SWRL

Inferred fact

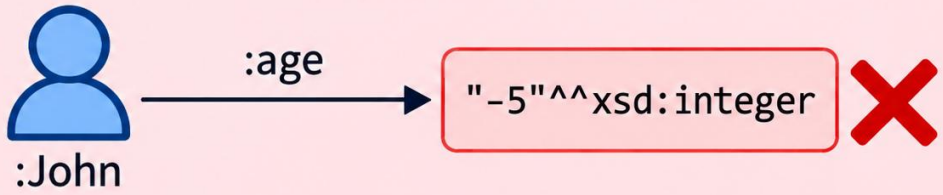
```
:John :hasMother :Mary .
```

Example (SWRL rule)

```
:Person(?x) ∧ :hasParent(?x, ?y) ∧ :Woman(?y)
  -> :hasMother(?x, ?y)
```

2. VALIDATION

identify RDF data violating constraints
(e.g., SHACL)



SHACL shape

```
:PersonShape a sh:NodeShape ;
  sh:targetClass :Person ;
  sh:property [
    sh:path :age ;
    sh:datatype xsd:integer ;
    sh:minInclusive 0 ;
  ] .
```

Validation result

Violation:

- Focus node: :John
- Path: :age
- Value: -5
- Constraint:
  sh:minInclusive 0

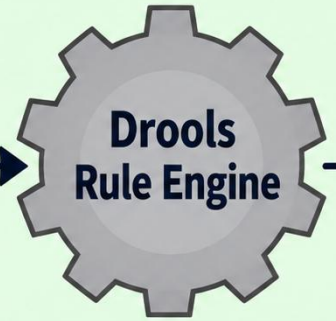
Four different roles of rules

3. DECISION

compute a business result (e.g., Drools)

Facts

```
:order123
:totalAmount 150 .
:customer :Alice .
```



Result

```
:order123
:discount 10 .
```

Example (Drools rule)

```
rule "VIP discount"
when
  $o : Order(totalAmount > 100, customer == "Alice")
then
  $o.setDiscount(10);
end
```

4. EVENT – CONDITION – ACTION

respond to a detected change (e.g., Home Assistant)



Motion detected in the hallway (event)

Condition

It is night time (condition)



Turn on the light (action)

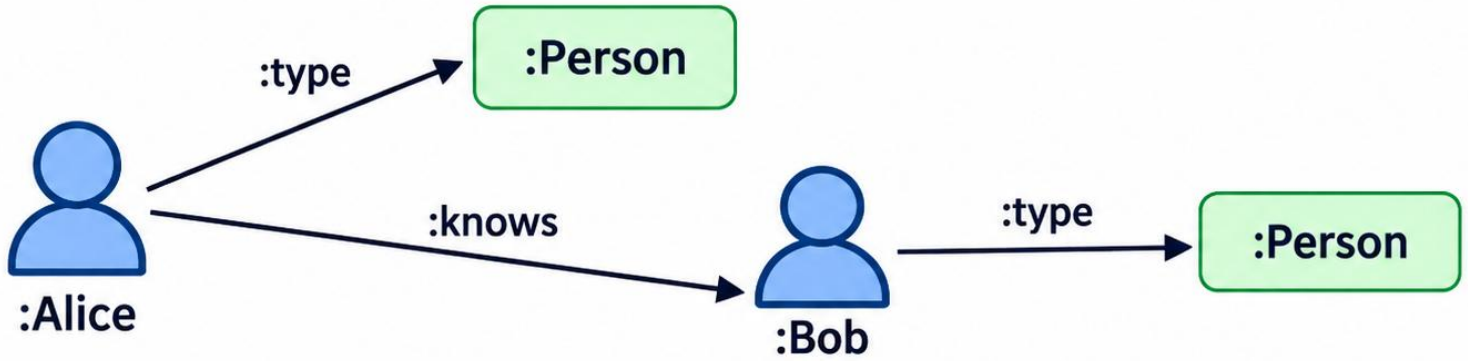
Example (Home Assistant automation)

```
alias: Hallway light at night
trigger:
  - platform: state
    entity_id: binary_sensor.hallway_motion
    to: "on"
condition:
  - condition: sun
    after: sunset
action:
  - service: light.turn_on
    target:
      entity_id: light.hallway
```

And SPARQL

5. SPARQL

query RDF graphs



Example (SPARQL query)

```
PREFIX : <http://example.org/>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>

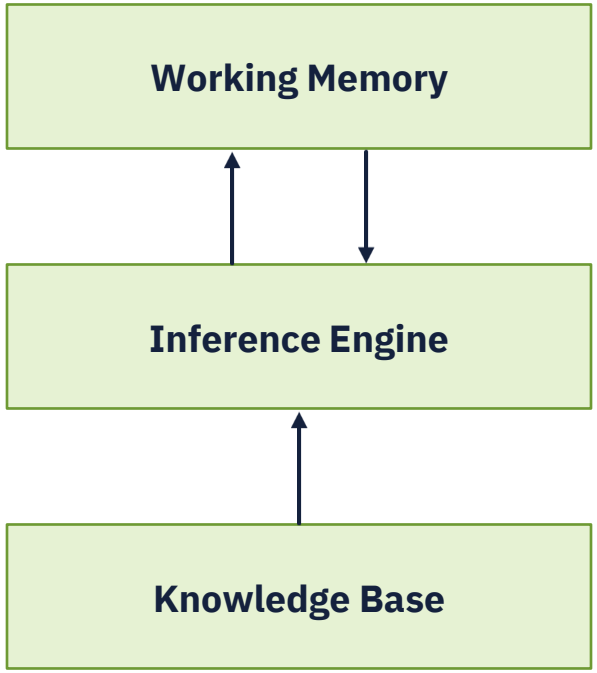
SELECT ?friend
WHERE {
   :Alice :knows ?friend .
   ?friend rdf:type :Person .
}
```

Result

friend
:Bob

Rule-based Systems

- A Rules Based System (RBS) is formed of:
  - A collection of **rules** - long term knowledge, domain-specific, stored in Knowledge Base
  - A collection of **facts** - short term knowledge, case-specific, stored in Working Memory
  - An **inference engine** – executes the rules



Reasoning in a rule-based system

- Two main tasks: **derive new facts**, or **determine whether a specific fact can be derived** from the rules and the known facts
- They imply two control regimes

Forward chaining · data-driven

Start with facts, determine applicable rules, apply one, repeat

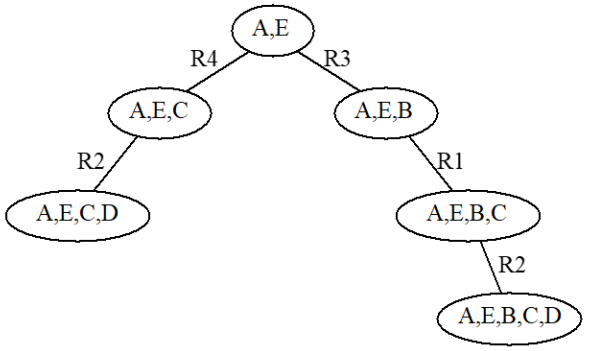
Drools, CLIPS, SWRL rule engines

Backward chaining · goal-oriented

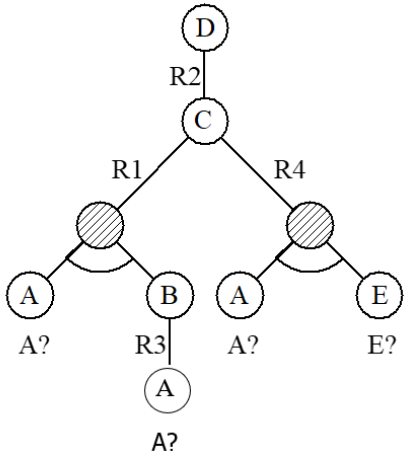
Look for rules whose conclusion matches the goal; solve the smaller subgoals

Prolog, MYCIN, Drools queries

- Is one better than the other? No general answer – it depends on the application



R1: **if** A&B **then** C
R2: **if** C **then** D
R3: **if** A **then** B
R4: **if** A&E **then** C

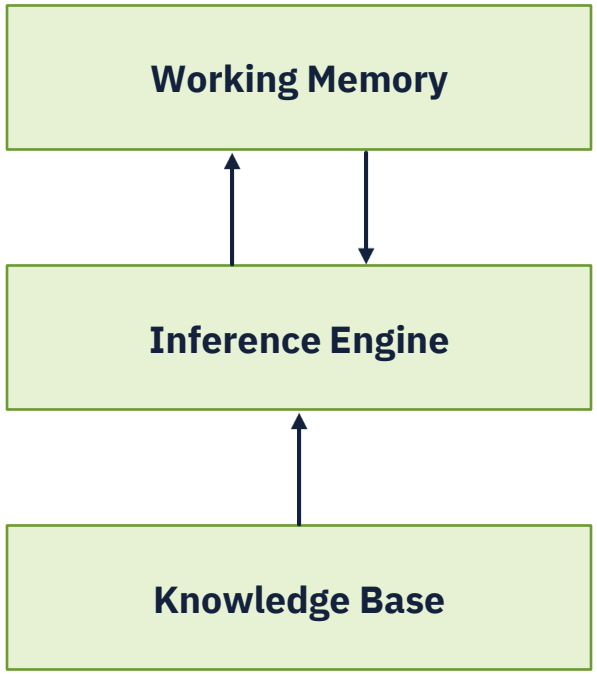


Working Memory

- The place where the **facts** (objects relevant for rules) are stored
- Some **operations** on the WM (there are many more)

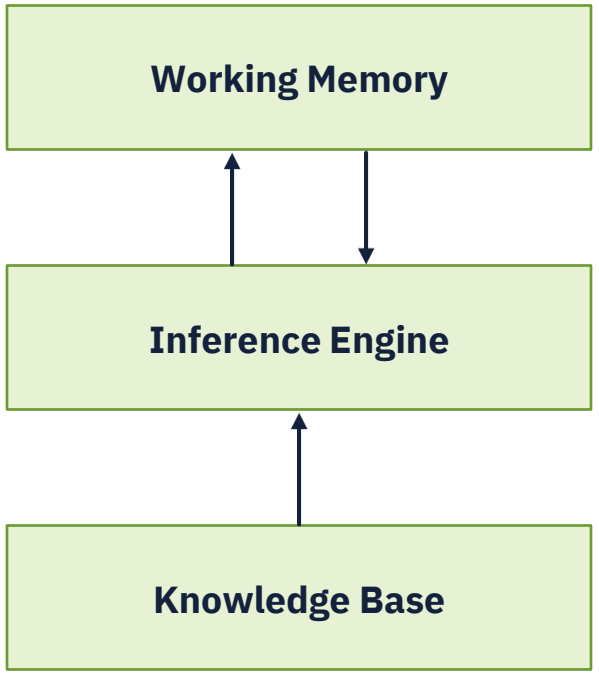
Operation	Meaning	In Drools
insert	put a new fact into the WM	insert
retract	delete a fact from the WM	delete (formerly retract)
update	change a fact already in the WM	update / modify

Every change to the WM is what drives the matching cycle



Inference engine

- Matches facts in the WM against rules
- **Matching** determines the applicable knowledge in the given situation
- Matching many rules against many facts is **computationally expensive** – special algorithms: **Rete** (Forgy) and, in Drools since v6, **Phreak**
- Matching often yields more than one applicable rule – they form the **conflict set** (Drools: the agenda)
- **Conflict resolution** picks one for firing
- **Firing**: executing the RHS of an applicable rule instance



How Rete matching works

- Rule conditions are compiled into a **network of test nodes –** dataflow network

Alpha nodes (intra-element tests) - **Alpha network**

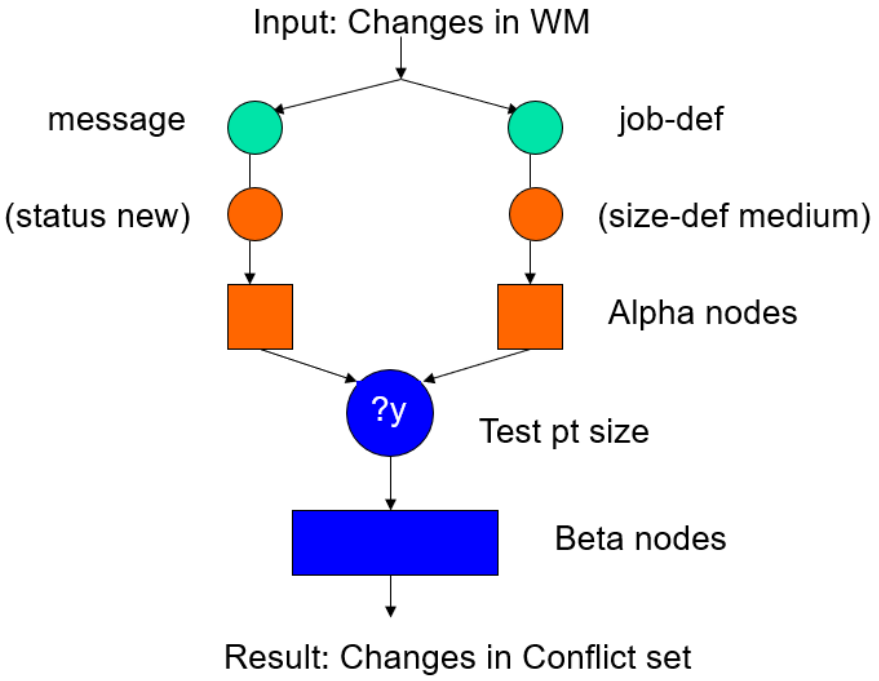
Beta nodes (inter-element tests) - **Beta network**

Terminal node: complete match → rule activation on the agenda

- Input – changes in WM
- WM changes: insert / update / delete of a fact
- Output – changes in Conflict Set / Agenda

- **Incremental**: partial matches are remembered, so only the changed facts are propagated – memory is traded for speed
- **Node sharing**: rules with common conditions share alpha and beta nodes
- **Phreak** (Drools ≥ 6): lazy, goal-oriented evolution of Rete – a rule's network is evaluated only when the rule may actually fire; scales to large rule bases

```
(defrule  cat-job-size
     (message (jobs  ?x) (status  new) (size ?y) )
     (job-def  (size-def  medium) (size ?y) )
=>
    (assert (job (job-name ?x) (job-size ?y))))
```



Conflict resolution

- Required when several rule instances are in the conflict set
- **Rule instance** = a reference to a rule + references to the WM objects that satisfy its conditions
- The standard strategy is **salience** = rule priority set by the knowledge engineer

Criterion	Meaning
Refractoriness	an instance that has fired does not fire again on the same facts
Recency	instances matching newer facts are preferred
Specificity	rules with more specific conditions in the LHS are preferred
Arbitrary	random or textual order as the final tie-breaker

In Drools: salience and agenda groups – otherwise do not rely on firing order

Rule-based languages and systems

Classic, very influential

OPS5 – Official Production System
MYCIN – medical expert system
CLIPS – C Language Integrated Production System, still maintained
Prolog – backward chaining, logic programming

Semantic Web

SWRL – Semantic Web Rule Language
RIF – Rule Interchange Format, W3C Recommendation
RuleML – Rule Markup Language, rule interchange
SHACL rules – rules over RDF graphs

Business rules

Drools – rule engine, now Apache KIE
DMN – Decision Model and Notation, OMG standard
Jess (Java Expert System Shell) – no longer actively developed

2. SWRL

- **SWRL = Semantic Web Rule Language**
- Proposed in 2004 as the rule language for OWL – a **W3C Member Submission**, never a W3C Recommendation
- All rules are expressed in terms of OWL concepts (classes, properties, individuals)
- Based on **RuleML** – Rule Markup Language, a markup language for publishing and sharing rule bases on the Web; an effort to standardize inference rules, focused on interoperation
- W3C's standard for rule interchange is **RIF** (2010); in practice SWRL remains the rule language of OWL tools – Protégé, Openllet, HermiT

SWRL rules

- A **SWRL rule** contains an antecedent part(*body)*, and a consequent (*head)* - positive conjunctions of *atoms*:

Atom , Atom ... -> Atom , Atom ....

BODY HEAD

- An **atom** is an expression of the form:

p(arg1, arg2,..., argn)

- ***P*** is a predicate symbol (OWL classes, properties or data types)
- ***arg1***, ***arg2***, ..., ***argn*** - arguments of the expression (OWL individuals or data values, or variables referring to them)

Atom types in SWRL

Atom type	OWL construct	Example
Class atom	owl:Class	Person(?x)
Individual property atom	owl:ObjectProperty	hasParent(?x, ?y)
Data valued property atom	owl:DatatypeProperty	hasAge(?x, ?age)
Different individuals atom	owl:differentFrom	differentFrom(?x, ?y)
Same individual atom	owl:sameAs	sameAs(?x, ?y)
Built-in atom	swrlb: built-ins	swrlb:greaterThan(?age, 18)
Data range atom	datatype / data range	xsd:integer(?age)

Built-ins cover comparisons, arithmetic, strings, dates and lists – e.g. swrlb:add, swrlb:stringConcat, swrlb:date

A SWRL rule

hasParent(?x, ?y) ^ hasBrother(?y, ?z) -> hasUncle(?x, ?z)

```
<!-- 2004 RuleML/SWRL XML syntax (abbreviated) -->
<ruleml:imp>
  <ruleml:_body>
    <swrlx:individualPropertyAtom swrlx:property="hasParent">
      <ruleml:var>x</ruleml:var> <ruleml:var>y</ruleml:var>
    </swrlx:individualPropertyAtom> <!-- … hasBrother(y, z) -->
  </ruleml:_body> <ruleml:_head> <!-- … hasUncle(x, z) --> </ruleml:_head>
</ruleml:imp>
```

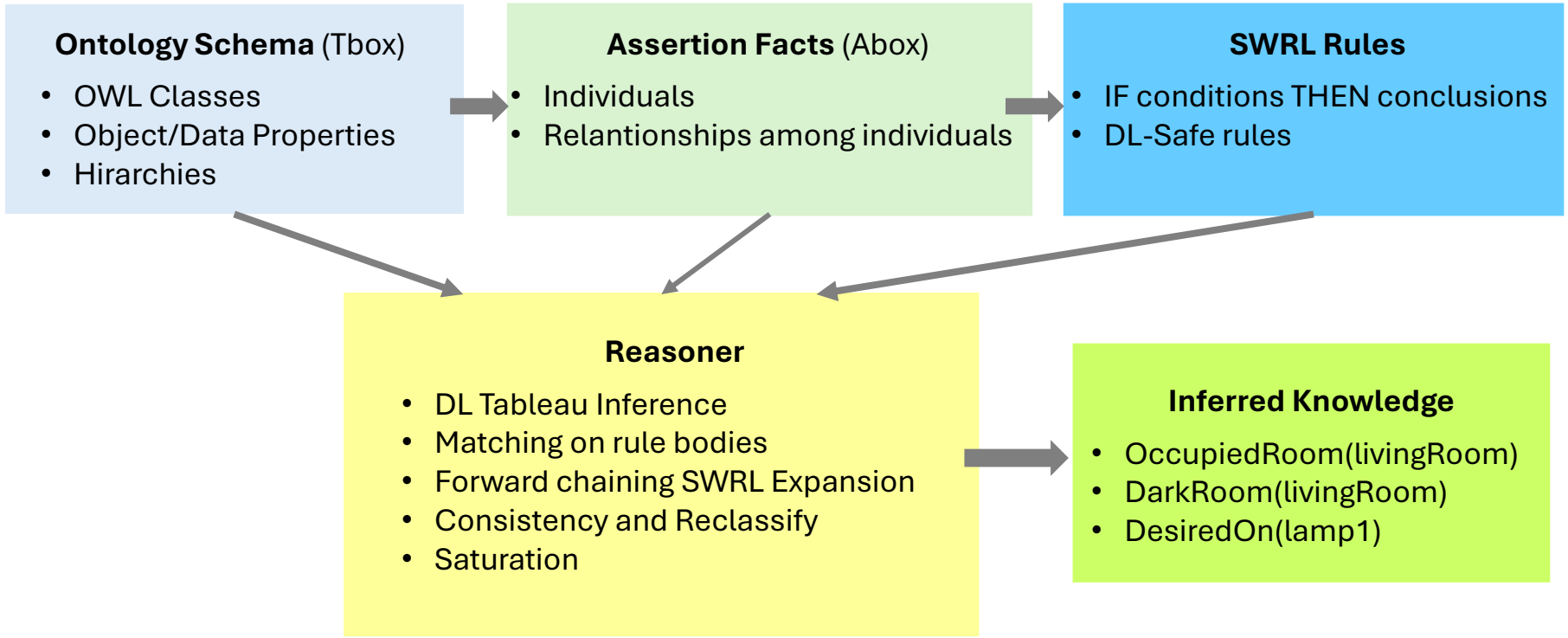
```
# OWL 2 functional syntax – how Protégé 5 stores the rule
DLSafeRule(
  Body( ObjectPropertyAtom(:hasParent  Variable(var:x) Variable(var:y))
        ObjectPropertyAtom(:hasBrother Variable(var:y) Variable(var:z)) )
  Head( ObjectPropertyAtom(:hasUncle   Variable(var:x) Variable(var:z)) ) )
```

Reasoning in OWL and SWRL

Engine	SWRL support	OBS
Pellet / Openllet	DL-safe rules, incl. built-ins	Openllet = maintained open-source of Pellet
HermiT	DL-safe rules, no built-ins	comes with Protégé 5
FaCT++, ELK	none – rules are ignored	fast for classification
SWRLTab / SWRLAPI	executes rules with a Drools-based engine	replaced the old Jess bridge (SWRLJessTab)

- The reasoner takes the ontology + SWRL rules and derives new knowledge
- Rules are applied to **named individuals**: when the body is satisfied, the head is added as an inferred fact or classification – forward-chaining behaviour

OWL/SWRL reasoning architecture



Pellet only supports **DL-safe SWRL rules**

Protégé 5.6 + SWRLTab

- **SWRLTab** – Protégé plugin to edit and execute SWRL rules; also runs **SQWRL** queries
- **SWRLAPI** – Java API for creating and running rules programmatically; Drools-based rule engine
- Rules are stored in the ontology and used by any SWRL-aware reasoner (Openllet, HermiT)

Rules – swrl_tutorial.owl

```
Person(?p) ^ bornOnDate(?p, ?d) ^ swrlb:date(?d, ?y, ?m, ?day, ?tz) -> bornInYear(?p, ?y)
Person(?p) ^ bornInYear(?p, ?y) ^ thisYear(?now) ^ swrlb:subtract(?age, ?now, ?y) -> hasAge(?p, ?age)
Person(?p) ^ hasAge(?p, ?age) ^ swrlb:greaterThanOrEqual(?age, 18) -> Adult(?p)
Person(?x) ^ (hasChild min 1 Person)(?x) -> Parent(?x)
```

OWL classes used in SWRL rules

SWRL supports the use of OWL class descriptions in rules

- Assert conclusions about individuals:

  Parent(?x) ->(hasChild >= 1)(?x)

  Publication(?p), (hasAuthor = 1)(?p) -> SingleAuthorPublication(?p)

- Find individuals that can match a rule:

  (hasChild >= 1)(?x) -> Parent(?x)

- The rule matches all individuals for which ***it can be proven*** that they are members of a class that has the specified cardinality restriction on a **hasChild** property – it classifies each such individual as a parent

DL-safe rules

- OWL DL + SWRL is not decidable any more
- **DL-Safe SWRL** rules are a restricted subset of SWRL rules.
- These rules have the desirable property of decidability
- Decidability is ensured by restricting rules to operate only on known individuals – **named individuals** - in an OWL ontology
- Binding to **individuals that are unknown** causes **undecidability**

DL-safe rules – example

TBox and ABox

Parent ≡ ∃hasChild.Person

Parent(a), Parent(b), Parent(c)
Person(d)
hasChild(a, d)

SWRL rule (DL-safe)

hasChild(?x, ?y) -> PersonWithChild(?x)

Inferred under DL-safety

PersonWithChild(a)

the rule only binds to individuals known in the ABox: **d** is a named child of **a**

Without the DL-safe restriction

PersonWithChild(a)
PersonWithChild(b)
PersonWithChild(c)

from Parent(b) and Parent ≡ ∃hasChild.Person, **b** must have ***some* child**, even if unnamed – same for **c**

What goes wrong without DL-safety?

- If rules could use all individuals, including anonymous ones, they could fire forever, creating new anonymous individuals again and again

1. hasChild(x, y) → PersonWithChild(x)
2. PersonWithChild(x) → ∃z hasChild(x, z) x has another child

The loop never ends → infinitely many children → infinitely many rule applications → reasoning does not terminate → **undecidable**

- Rule 2 is an **existential rule** it cannot be written in SWRL at all
- But OWL axioms like Parent ≡ ∃hasChild.Person have the same effect inside the reasoner
- With DL-safe rules, rule 1 never binds to the anonymous z – no loop
- With DL-safe rules, rule 2 **cannot fire** unless there is a **named individual z** already in the ABox

Example with Home Assistant

- **Home Assistant** – a popular open-source home automation platform: control smart devices, automate routines, integrate many services, locally or in the cloud

https://www.home-assistant.io/

- Used for smart lighting, home security, energy saving, voice control, integration of different devices
- Automations follow a simple **trigger → condition → action** scheme:

Trigger	**Condition (optional)**	**Action**
what starts the automation – e.g. motion detected	extra check before running – e.g. only at night	what happens – e.g. turn on a lamp

Form of the rule: “If situation X happens and Y is true, then do Z” – an event-condition-action rule

Home Assistant automations in YAML

```
alias: Turn on hallway light at night
triggers:
  - trigger: state
    entity_id: binary_sensor.motion
    to: "on"
conditions:
  - condition: time
    after: "21:00:00"
actions:
  - action: light.turn_on
    target:
      entity_id: light.hallway
```

Declarative rules in **YAML** - data serialization format. Current syntax (since release 2024.10)

Key words
triggers:
trigger: state
conditions:
actions:
action: ...

“When motion is detected in the hallway after 9:00 PM, turn on the hallway light”

Semantic automation in Home Assistant

What if, instead of rigid rules like

> If motion in hallway → turn on the light in the hallway

we want reusable, **knowledge-driven** rules like:

> **If Room is occupied AND Room is dark → illuminate(Room)**

We use rules to describe **semantic automation** – the home understands concepts like:

- “A bedroom is a type of room”
- “Lamps are lights → they provide illumination”
- “If it’s dark and someone is in a room → turn on a light IN THAT ROOM (any light, not a specific ID)”

A minimal OWL ontology for the home

Classes (Manchester syntax)

```
Ontology: <http://homeassistant/kg>
Class: Device
Class: Sensor        SubClassOf: Device
Class: MotionSensor  SubClassOf: Sensor
Class: LuxSensor     SubClassOf: Sensor
Class: WaterLeakSensor SubClassOf: Sensor
Class: Light         SubClassOf: Device
Class: Valve         SubClassOf: Device
Class: Room
Class: OccupiedRoom  SubClassOf: Room
Class: DarkRoom      SubClassOf: Room
Class: EmptyRoom     SubClassOf: Room
```

One individual per Home Assistant entity

```
Individual: livingRoom
  Types: Room
Individual: motion1
  Types: MotionSensor
  Facts: locatedIn livingRoom, hasState "on"
Individual: lux1
  Types: LuxSensor
  Facts: locatedIn livingRoom, measuredLux 12
Individual: lamp1
  Types: Light
  Facts: locatedIn livingRoom, hasState "off"
```

Each entity (sensor, light, valve) is linked to its room by ***locatedIn***

Intent classes and properties

"Intent" marker classes (HA will act on these)
Class: DesiredOn
Class: DesiredOff
Class: DesiredClose
Class: SuppressedAction
Object properties
ObjectProperty: locatedIn
Domain: owl:Thing
Range: Room
Data properties
DataProperty: hasState **Range**: xsd:string # e.g., "on", "off", "quiet"
DataProperty: measuredLux **Range**: xsd:decimal
DataProperty: lastMotionMinutes **Range**: xsd:integer

SWRL rules: occupancy and darkness

Add an individual per entity (e.g., livingRoom : Room, lamp1 : Light, motion1 : MotionSensor, etc.), each linked by **locatedIn**

SWRL rules

Occupancy & darkness → need illumination

Motion ON ⇒ room is occupied

MotionSensor(?s) ^ hasState(?s, "on") ^ locatedIn(?s, ?r)

-> OccupiedRoom(?r)

Lux below threshold ⇒ room is dark

LuxSensor(?ls) ^ measuredLux(?ls, ?lx) ^ swrlb:lessThan(?lx, 20) ^ locatedIn(?ls, ?r)

-> DarkRoom(?r)

If a room is occupied and dark ⇒ any Light in that room should be ON

OccupiedRoom(?r) ^ DarkRoom(?r) ^ Light(?l) ^ locatedIn(?l, ?r)

-> DesiredOn(?l)

SWRL rules: quiet hours and energy saving

Quiet hours / sleep mode → suppress lighting

If room is "quiet" (e.g., house night mode), suppress any DesiredOn in that room

Room(?r) ^ hasState(?r, "quiet") ^ DesiredOn(?l) ^ locatedIn(?l, ?r)
 -> SuppressedAction(?l)

Energy saving: empty rooms → turn lights off

No motion for > 30 minutes ⇒ room is empty

Room(?r) ^ lastMotionMinutes(?r, ?m) ^ swrlb:greaterThan(?m, 30)
 -> EmptyRoom(?r)

If room is empty and a light there is ON ⇒ we desire it OFF

EmptyRoom(?r) ^ Light(?l) ^ locatedIn(?l, ?r) ^ hasState(?l, "on")
 -> DesiredOff(?l)

Semantic automation – inference at work

For example, with the rules

MotionSensor(?s) ^ hasState(?s,"on") ^ locatedIn(?s,?r) -> OccupiedRoom(?r)

LuxSensor(?s) ^ measuredLux(?s,?lx) ^ swrlb:lessThan(?lx,20) ^ locatedIn(?s,?r)
-> DarkRoom(?r)

OccupiedRoom(?r) ^ DarkRoom(?r) ^ Light(?l) ^ locatedIn(?l,?r) -> DesiredOn(?l)

The systems infers

OccupiedRoom(livingRoom)

DarkRoom(livingRoom)

DesiredOn(lamp1)

Home Assistant reads "lamp1" is of type DesiredOn → therefore, **turn it on**

SWRL is monotonic

OWL + SWRL reason under the open-world assumption and **never retract** a conclusion. The previous rules show it:

- OccupiedRoom and EmptyRoom can both hold for the same room (stale motion state)
- DesiredOn and SuppressedAction both hold for lamp1 – who wins?
- When motion stops, nothing is withdrawn within the same reasoning run
- No negation as failure: “no sensor in the room is on” cannot be stated

Solution

- Declare OccupiedRoom and EmptyRoom disjoint – the reasoner then reports the inconsistency
- Let the designer apply a priority: SuppressedAction overrides DesiredOn
- Rebuild the ABox from the current state and re-run the reasoner at each change
- Or move control logic to a production-rule engine with retraction and negation → **Drools**

Aligning with SAREF (ETSI)

Reuse a standard ontology: **SAREF** – Smart Applications REFerence ontology (saref.etsi.org), with its building extension **SAREF4BLDG**

```
Prefix: ha:    <http://homeassistant/kg#>
Prefix: saref:  <https://saref.etsi.org/core/>
Prefix: s4bldg: <https://saref.etsi.org/saref4bldg/>

Class: ha:Room       SubClassOf: s4bldg:BuildingSpace
Class: ha:Device     SubClassOf: saref:Device
Class: ha:Light      SubClassOf: ha:Device, saref:Actuator
Class: ha:MotionSensor SubClassOf: ha:Device, saref:Sensor
Class: ha:LuxSensor    SubClassOf: ha:Device, saref:Sensor
Class: ha:OccupiedRoom SubClassOf: ha:Room
Class: ha:DarkRoom     SubClassOf: ha:Room
```

```
ObjectProperty: ha:locatedIn
  InverseOf: s4bldg:contains
DataProperty: ha:hasState
  Range: xsd:string
DataProperty: ha:measuredLux
  Range: xsd:decimal

Individual: ha:lamp1
  Types: ha:Light
  Facts: ha:locatedIn ha:livingRoom,
       ha:hasState "off"
```

BuildingSpace and isLocatedIn moved from SAREF core (v3) to SAREF4BLDG

Align with SubClassOf, not EquivalentTo – a lamp is not a light switch

SAREF models states as saref:State individuals: keep simple data properties in ha:

3. DROOLS

- **Drools** – open-source **business rules management system** and **rule engine** for Java
- Since 2023 part of **Apache KIE** (incubating) at the Apache Software Foundation, with jBPM, OptaPlanner and Kogito; Apache 2.0 licence; current release 10.2.0 (April 2026)

Drools engine
rule execution, Phreak algorithm

DRL
Drools Rule Language – rule authoring

Decision tables & DMN
spreadsheets and OMG decision models

Drools Fusion
complex event processing, temporal reasoning

Rule Units
modern programming model for new projects

Kogito
cloud-native decision services on Quarkus / Spring Boot

Examples of Drools rules

```
package com.example.rules.customer      // organizes rules
import com.example.model.Customer;      // Java model classes

rule "Loyal customer discount"          // a business rule
   salience 10                          // rule priority
when                                    // condition (LHS)
   $c : Customer( loyaltyPoints > 1000 )
then                                    // action (RHS)
   modify( $c ) { setDiscount( 10 ) };
   System.out.println( "Applied 10% discount for loyal customer: " + $c.getName() );
end
```

$c – variable bound to the matched Customer object

modify – tells the engine the fact changed; a plain setter is invisible to it

Examples of Drools rules: fraud check

```
package com.example.rules.transaction
import com.example.model.Transaction;

rule "Flag high transaction"
   salience 5
when
   $t : Transaction( amount > 5000, location == "International" )
then
   modify( $t ) { setFlagged( true ) };
   System.out.println( "Transaction flagged for review: " + $t.getId() );
end
```

Comma-separated constraints inside a pattern are a conjunction

Examples of Drools rules: insurance

```
package com.example.rules.insurance
import com.example.model.Driver;

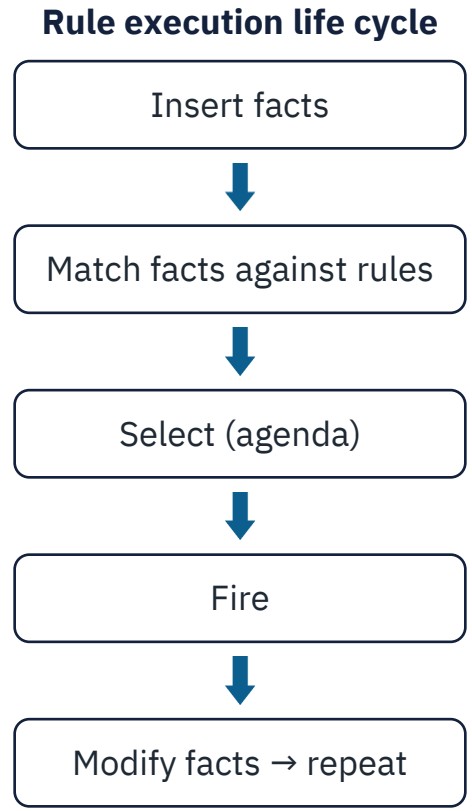
rule "High-risk driver surcharge"
   no-loop true
when
   $d : Driver( age < 25, accidents > 1 )
then
   modify( $d ) { setPremium( $d.getPremium() * 1.25 ) };
   System.out.println( "Applied high-risk surcharge to driver: " + $d.getName() );
end
```

Inference in Drools

- A typical rule-based system
- **Forward chaining** engine based on **Phreak**, an evolution of Rete
- Follows a **Match → Select → Execute** cycle

Write business rules in the Drools Rule Language (.drl)

Runtime – rule execution: create a **KieSession** (or a Rule Unit instance) to hold the facts, then let the engine match and execute the rules



KIE and Drools

KIE stands for **Knowledge Is Everything** – the framework and API used by Drools to manage, load and execute business rules, decision models and workflows

Drools needed a standard way to organize and execute business knowledge – rules, decisions and processes.

KIE provides this foundation, and since 2023 it is also the name of the Apache project that hosts Drools, jBPM, OptaPlanner and Kogito.

Developers and business users work with

- Rule files (.drl)
- Decision tables (.xlsx)
- DMN decision models (.dmn)
- Business processes (.bpmn) – jBPM
- Java Rule Units and rule flows

Drools components

- **Knowledge Base** (KieBase) – manages a collection of rules; typically all rules of a package, or a list of packages, form a knowledge base
- **Knowledge Session** (KieSession) – a runtime instance that loads a knowledge base; facts are inserted into it at runtime

Stateless session

one call: insert facts, fire, discard; it does not track state changes – rules newly enabled by a change are not triggered

Stateful session

long-lived; updates to facts are tracked – on any change, all rules are rechecked and fired again if necessary

By default, Drools sessions are stateful – the smart-home case needs exactly this

Drools elements

Element	Purpose
salience	order in which rules fire – higher salience, higher priority
agenda-group	alternative way to control order; salience applies within a group; default group is “MAIN”
activation-group	at most one rule of the group fires – the one with the highest salience; the others are skipped
no-loop	a rule does not re-activate because of its own modification
insertLogical	truth maintenance: the inserted fact is retracted automatically when the rule’s conditions stop holding
not / exists	negation as failure / existence over working memory – closed-world tests
property reactivity	only patterns that test a changed property are re-evaluated (default since Drools 7)

An example: customer discounts

Calculate discounts for customers based on their loyalty level and total purchase amount.

- If the customer is **VIP**, give a **20%** discount
- If the customer spent **more than $500**, give a **10%** discount
- If the customer has **more than 5 loyalty years**, give a **5%** discount
- Discounts should **not exceed 20%** in total

Drools rules (discount-rules.drl) 1/2

```
package com.example.rules
import com.example.model.Customer;

rule "VIP discount"
when
   $c : Customer( vip == true )      // $c binds to the Customer object
then
   modify( $c ) { setDiscount( $c.getDiscount() + 20 ) };
   System.out.println( "Rule fired: VIP discount applied" );
end

rule "High purchase discount"
when
   $c : Customer( totalPurchase > 500 )
then
   modify( $c ) { setDiscount( $c.getDiscount() + 10 ) };
   System.out.println( "Rule fired: High purchase discount applied" );
end
```

Drools rules (discount-rules.drl) 2/2

```
rule "Loyal customer discount"
when
   $c : Customer( loyaltyYears > 5 )
then
   modify( $c ) { setDiscount( $c.getDiscount() + 5 ) };
   System.out.println( "Rule fired: loyal discount" );
end

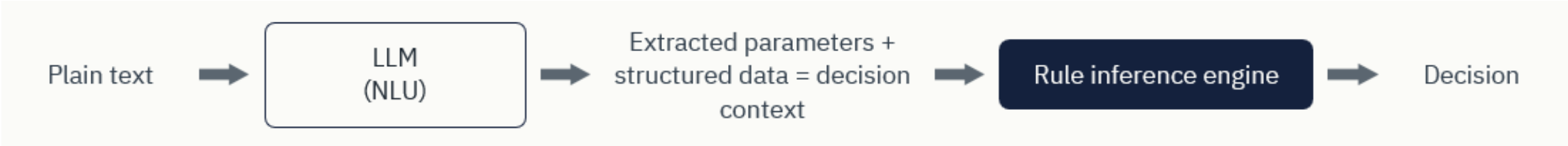
rule "Cap discount at 20%"  // the business cap, now a rule
when
   $c : Customer( discount > 20 )
then
   modify( $c ) { setDiscount( 20 ) };
end
```

```
rule "Report final discount"
   salience -10
when
   $c : Customer()
then
   System.out.println( "Final: "
      + $c.getDiscount() + "%" );
end
```

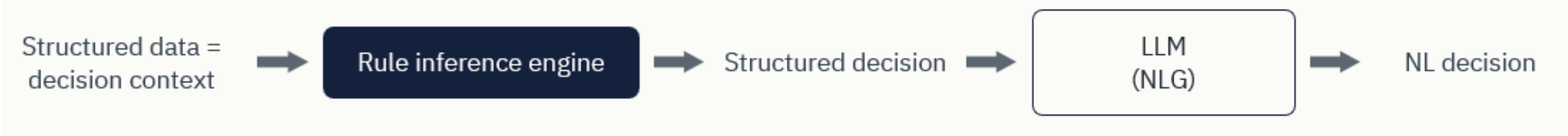
4. Rules and LLMs

Rules and LLMs: augmenting automation

LLM followed by rule reasoning – the LLM comprehends documents/text and extracts structured data; a rule engine then reasons **deterministically** about it



Rule reasoning followed by LLM – the rule engine first makes a decision on structured data; an LLM then generates a natural-language outcome from the decision

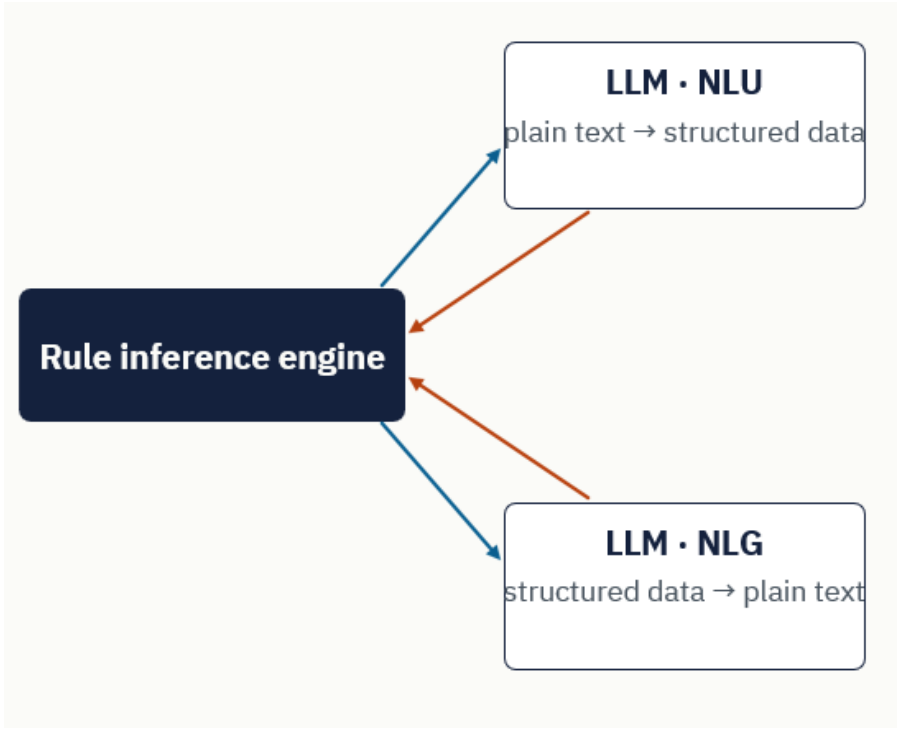


Rule reasoning combined with an LLM

The **rule inference engine** acts as the master engine, invoking an LLM

The rule engine drives the rule evaluation and dynamically calls the LLM for two delegated tasks:

- Processing text received in its decision context – understanding it and extracting structured data
- Generating text – a summary of the automated decision, or an explanation



Rules bring reliable reasoning to a chatbot

- The **LLM drives the conversational experience**; it gives input to a rule-based decision engine that applies the business decisions
- The chatbot must recognize, during the conversation, **when to trigger** a decision service
- It guides the dialog to collect the context and invokes the decision engine **when all facts are set**
- The engine returns results, which are given back in the conversation as text



References and resources

- SWRL – W3C Member Submission (2004): w3.org/submissions/SWRL
- RIF Overview – W3C Recommendation: w3.org/TR/rif-overview
- SPARQL 1.1 Query Language: w3.org/TR/sparql11-query
- SPARQL 1.2 / RDF 1.2 drafts: w3.org/TR/rdf12-concepts
- SAREF and SAREF4BLDG (ETSI): saref.etsi.org
- Protégé: protege.stanford.edu
- SWRLTab / SWRLAPI: github.com/protegeproject/swrltab
- Openllet: github.com/Galigator/openllet
- Apache KIE (Drools): kie.apache.org
- Home Assistant automations: home-assistant.io/docs/automation/yaml